

```
    component: Home,
    title: 'Home page',
    canActivate: [authGuard],
  },
  {
    path: 'details/:id',
    component: Details,
    title: 'Details',
    canActivate: [authGuard],
  },
];
```

JWT EN LA AUTENTICACIÓN:

- El backend devuelve un JWT al hacer login
- Se guarda el token en localStorage (en lugar del username)
- Se envía el token en el header Authorization: Bearer <token> en cada petición
- Se decodifica el token para obtener info del usuario

A. Pruebas en Angular

INTRODUCCIÓN

En este anexo veremos de forma básica el concepto de pruebas automáticas en aplicaciones Angular. En este contexto, las pruebas pueden ser de distintos tipos:

- **Pruebas unitarias:** Verifican el funcionamiento de una parte concreta de la aplicación de forma aislada, como una función, un servicio o un componente. No dependen de servicios externos ni de otros módulos de la aplicación. Son rápidas de ejecutar y permiten detectar errores en fases tempranas del desarrollo.
- **Pruebas de integración:** Comprueban que varias partes de la aplicación funcionan correctamente juntas, por ejemplo, un componente que utiliza un servicio o la comunicación entre varios componentes. Aunque son más completas que las pruebas unitarias, siguen ejecutándose en un entorno controlado.
- **Pruebas de extremo a extremo (E2E):** Simulan el comportamiento real de un usuario en la aplicación, interactuando con la interfaz (clics, formularios, navegación). Validan flujos completos desde el inicio hasta el final, pero son más lentas y costosas de mantener.

En esta parte, **nos centraremos exclusivamente en las pruebas unitarias**, ya que permiten verificar el correcto funcionamiento de componentes, servicios y lógica pura de forma aislada, rápida y sin depender de servicios externos ni del navegador real.

El objetivo no es cubrir todos los casos posibles, sino comprender cómo validar que una parte

concreta de la aplicación se comporta como se espera.

Cambios en el sistema de pruebas en Angular moderno - **Novedad**

NOTE: This guide covers the default testing setup for new Angular CLI projects, which uses Vitest. If you are migrating an existing project from Karma, see the [Migrating from Karma to Vitest guide](#). Karma is still supported; for more information, see the [Karma testing guide](#).

Las versiones recientes de Angular han actualizado el sistema de pruebas por defecto, pasando a utilizar **Vitest** en los proyectos nuevos creados con Angular CLI. Este cambio simplifica la ejecución de tests y reduce la configuración necesaria, especialmente en aplicaciones basadas en componentes standalone.

El sistema anterior, basado en **Karma y Jasmine**, sigue siendo compatible y puede encontrarse en muchos proyectos existentes, existiendo además guías oficiales de migración: <https://angular.dev/guide/testing/migrating-to-vitest>

PRINCIPIOS BÁSICOS DE TESTING UNITARIOS EN ANGULAR

- a. Lógica pura
 - Funciones que no dependen de Angular.
 - Se prueban directamente usando describe, it y expect.
- b. Componentes y servicios
 - Se prueban de forma aislada.
 - En componentes se comprueba también lo que se renderiza en el HTML (DOM).
- c. TestBed (configuración ligera)
 - TestBed crea un Angular pequeño solo para el test que permite crear componentes y servicios para los tests.
 - Solo se importan el componente y las dependencias necesarias.
- d. Mocks (simulación)
 - Se simulan comportamientos para controlar el resultado del test.
 - Las peticiones HTTP no deben llamar a Internet.
 - El router no debe navegar realmente.

CARACTERÍSTICAS DE UNA BUENA PRUEBA UNITARIA

Una prueba unitaria debe cumplir una serie de características que faciliten su uso y mantenimiento a lo largo del tiempo. En esta práctica se tendrán en cuenta las siguientes:

- a. Fáciles de escribir: Una prueba debe poder escribirse sin una configuración compleja. Si una prueba requiere demasiado código o demasiadas dependencias, es probable que no esté bien planteada.

- b. Fáciles de leer: El código de una prueba debe entenderse casi como una explicación de lo que se está comprobando. Al leer una prueba debería quedar claro qué se está probando y cuál es el resultado esperado.
- c. Rápidas: Las pruebas unitarias deben ejecutarse en poco tiempo. Por este motivo no deben depender de Internet, del sistema de archivos ni de temporizadores reales.
- d. Flexibles (fáciles de mantener): Una prueba debe fallar solo cuando el comportamiento que se está probando cambia realmente. Cambios menores en la implementación no deberían obligar a reescribir todas las pruebas.

Una forma sencilla de escribir pruebas claras y legibles es seguir el patrón **AAA**, que divide cada prueba en tres partes:

- **Arrange (Preparar):** Se prepara el entorno del test: datos de entrada, objetos necesarios o mocks.
- **Act (Actuar):** Se ejecuta la acción que se quiere probar (llamar a una función, método o evento).
- **Assert (Comprobar):** Se verifica que el resultado obtenido es el esperado mediante `expect(...)`.

CONCEPTO DE COBERTURA (COVERAGE)

La cobertura indica qué parte del código ha sido ejecutada al lanzar las pruebas automáticas. Se expresa normalmente en porcentajes y **sirve como orientación, no como garantía de calidad**. La cobertura ayuda a detectar qué partes del código no se están probando.

Tipos habituales de cobertura:

- a. Cobertura de líneas: Porcentaje de líneas de código que se han ejecutado al menos una vez durante los tests.
- b. Cobertura de ramas: Porcentaje de decisiones (if, else, switch) que se han probado en sus distintos caminos.
- c. Cobertura de funciones: Porcentaje de funciones o métodos que han sido invocados en las pruebas.
- d. Cobertura de condiciones: Porcentaje de condiciones lógicas que se han evaluado tanto como verdaderas como falsas.

Importante:

- Una alta cobertura no garantiza que el código esté bien probado.
- Una cobertura baja no implica que el código sea incorrecto.
- La cobertura es una herramienta de apoyo, no un objetivo en sí mismo.

PRIMER TEST

Vamos a hacer unas pruebas sobre la aplicación **MI_APP_03**

Primero comprueba que tu proyecto Angular está configurado por defecto para usar VITEST:

```
package.json > ...
35   "devDependencies": {
36     "@angular/build": "^21.1.0",
37     "@angular/cli": "^21.1.0",
38     "@angular/compiler-cli": "^21.1.0",
39     "jsdom": "^27.1.0",
40     "typescript": "~5.9.2",
41     "vitest": "^4.0.8"
42   }
43 }
```

```
tsconfig.spec.json > ...
2   /* To learn more about Angular compilation
3   {
4     "extends": "./tsconfig.json",
5     "compilerOptions": {
6       "outDir": "./out-tsc/spec",
7       "types": [
8         "vitest/globals"
9       ]
10    },
11    "include": [
12      "src/**/*.d.ts",
13      "src/**/*.ts"
14    ]
15  }
16
```

Después borra todos los ficheros **.spect.ts** **menos el fichero app.spect.ts** Analiza su contenido por defecto y modificarlo para que quede así:

app.spect.ts

```
import { TestBed } from '@angular/core/testing';
import { App } from './app';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
    }).compileComponents();
  });
});
```

```
});

it('should be true', () => {
  expect(true).toBe(true);
});
});
```

- **describe** se utiliza para agrupar un conjunto de pruebas relacionadas, normalmente asociadas a un componente o servicio concreto; su primer parámetro es una descripción textual que ayuda a identificar qué se está probando.
- **beforeEach** (lo genera el asistente por defecto) se ejecuta antes de cada prueba, y en él se prepara el entorno de testing mediante TestBed, que crea una versión ligera de Angular donde se cargan los componentes necesarios. Se encarga de preparar un escenario limpio antes de cada test, garantizando que lo que ocurra en una prueba no contamine a la siguiente. Sin el beforeEach, si un test modifica una variable global o una propiedad del componente, el siguiente test heredaría ese estado.
- **it** define una prueba individual, que comprueba una condición concreta mediante una aserción con **expect(...)**.

Nota: El enfoque varía un poco si usamos standalone true o standalone false

```
beforeEach(async () => {
  await
  TestBed.configureTestingModule({
    imports:
    [RouterModule.forRoot([])],
    declarations: [App],
  }).compileComponents();
});
```

```
beforeEach(async () => {
  await
  TestBed.configureTestingModule({
    imports: [AppComponent],
  }).compileComponents();
});
```

Finalmente lanza los test de prueba de la aplicación con: `ng test`

```
~/work/fw_ejercicios_m/mi_app_03 > npm run test -- --watch
ng test
Initial chunk files | Names | Raw size
chunk-XSGC6WRG.js | - | 4.05 kB
spec-app-app.js | spec-app-app | 2.32 kB
init-testbed.js | init-testbed | 1.20 kB
styles.css | styles | 584 bytes
| Initial total | 8.16 kB

Application bundle generation complete. [0.588 seconds] - 2026-02-08T17:11:08.466Z
Watch mode enabled. Watching for file changes...

DEV v4.0.16 /Users/alberto/workspace/fw_ejercicios_montoro_alberto_25-26/mi_app_03
✓ mi_app_03 src/app/app.spec.ts (1 test) 1ms
  ✓ App (1)
    ✓ should be true 1ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 18:11:08
Duration 433ms (transform 22ms, setup 89ms, import 29ms, tests 1ms, environment 251ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Al ejecutarlo, el proceso de testing queda en ejecución continua y se mantiene a la espera de cambios. Esto permite ir definiendo o modificando pruebas y, al guardar los ficheros, ver automáticamente los resultados por pantalla sin necesidad de relanzar el comando.

```
PASS Waiting for file changes...
press h to show help, press q to quit

Watch Usage
press a, return to rerun all tests
press r to rerun current pattern tests
press f to rerun only failed tests
press u to update snapshot
press p to filter by a filename
press t to filter by a test name regex pattern
press w to filter by a project name
press b to start the browser server if not started yet
press q to quit
```

Modifica el test:

```
...
it('should be true', () => {
  expect(false).toBe(true);
});
```

```
> mi_app_03 src/app/app.spec.ts (1 test | 1 failed) 3ms
> App (1)
  ✕ should be true 3ms

----- Failed Tests 1 -----

FAIL mi_app_03 src/app/app.spec.ts > App > should be true
AssertionError: expected false to be true // Object.is equality

- Expected
+ Received

- true
+ false

> src/app/app.spec.ts:12:19
   10|
   11|     it('should be true', () => {
   12|       expect(false).toBe(true);
      |                       ^
   13|     });
   14|   });
```

```
Test Files  1 failed (1)
Tests       1 failed (1)
Start at    18:18:23
Duration    128ms
```

```
FAIL Tests failed. Watching for file changes...
press h to show help, press q to quit
```

Añade dos nuevos test:

```
...
it('should be 4', () => {
  expect(2+2).toBe(4);
});

it('should not divide by 0', () => {
  let a = 2;
  let b = 1;
  let result = a / b;
  expect(result).not.toBe(Infinity);
});
```

```
> mi_app_03 src/app/app.spec.ts (3 tests | 1 failed) 4ms
> App (3)
  ✗ should be true 3ms
  ✓ should be 4 0ms
  ✓ should not divide by 0 0ms

----- Failed Tests 1 -----

FAIL mi_app_03 src/app/app.spec.ts > App > should be true
AssertionError: expected false to be true // Object.is equality
```

Añade otro test:

```
...
it('should stop test execution when an expect fails', () => {
  const a = 2;
  const b = 2;

  // Este expect es correcto
  expect(a - b).toBe(0);

  // Este expect FALLA
  expect(a + b).toBe(5);

  // Este expect NO se ejecuta nunca
  expect(true).toBe(true);
});
```

```
> mi_app_03 src/app/app.spec.ts (4 tests | 2 failed) 4ms
> App (4)
  ✗ should be true 3ms
  ✓ should be 4 0ms
  ✓ should not divide by 0 0ms
  ✗ should stop test execution when an expect fails 0ms

----- Failed Tests 2 -----
```

Nota: En algunos casos, tras modificar un test, aunque se pulse 'a' (ejecutar todos) o 'r' (repetir), el comportamiento puede no ser el esperado. Si ocurre esto, pulsa 'q' para salir del modo interactivo y vuelve a lanzar de nuevo todos los tests ejecutando otra vez: `ng test`

TEST BÁSICOS EN COMPONENTES

Vamos a probar el componente app y para ello vamos a ampliarlo ligeramente.

```
<main>
  <a [routerLink]="['/']">
    <header class="brand-name">
      
    </header>
  </a>

  <section class="content">
    <router-outlet></router-outlet> <!-- Ahora sí es útil -->
  </section>
  <section class="aux" [ngStyle]="auxStyle">
    <button class="btn-test" (click)="incrementClicks()">Click me</button>
    <button class="btn-test" (click)="incrementClicks(2)">Click me +2</button>

    <p class="click-counter">
      Clicks: {{ clicks }}
    </p>
  </section>
</main>
```

```
import { Component, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [CommonModule, RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('mi_app_03');
  public subTitle = signal('');
  public year = 2025;
  public clicks = 0;

  public auxStyle: Record<string, string> = {
    textAlign: 'center',
    marginTop: '100px',
  };

  constructor() {
    this.subTitle.set('DWEK & Frameworks');
    this.year = new Date().getFullYear();
  }

  public showTitleAndsubTitle() {
    return `${this.title()} - ${this.subTitle()} [${this.year}]`;
  }

  public incrementClicks(inc: number = 1) {
    this.clicks=this.clicks+inc;
  }
}
```

```
}  
}
```

En el mismo fichero `app.spect.ts`, primero comenta los test anteriores y luego añade estos test:

```
// El componente app importa RouterOutlet y usas routerLink  
// En los tests conviene añadir provideRouter([])  
  
import { provideRouter } from '@angular/router';  
  
...  
describe('App', () => {  
  beforeEach(async () => {  
    await TestBed.configureTestingModule({  
      imports: [App],  
      providers: [provideRouter([])],  
    }).compileComponents();  
  });  
  
  ...  
  it('should create the app', () => {  
    const fixture = TestBed.createComponent(App);  
    const app = fixture.componentInstance;  
    expect(app).toBeTruthy();  
  });  
});
```

- Se instancia el componente y devuelve un fixture, que representa el contenedor del componente en el test.
 - Se ejecuta el constructor del componente. NO se ejecuta `ngOnInit` automáticamente en este test.
- A partir de este fixture se accede a `componentInstance`, que es la instancia real de la clase `App`.
- Crear el componente no es lo mismo que renderizarlo.
- La aserción `expect(app).toBeTruthy()` verifica que dicha instancia existe y no es `null` ni `undefined`.

Ahora añade este test:

```
...
it('should keep the title and subTitle unchanged', () => {
  const fixture = TestBed.createComponent(App);
  const app = fixture.componentInstance;

  app.subTitle.set('IES Gregorio Prieto');

  console.log('>>>Privado:', app['title']());
  console.log('>>>Publico:', app.subTitle());
  console.log('>>>Publico:', app.year);
  console.log('>>>Publico:', app.showTitleAndsubTitle());

  expect(app.year).toBe(new Date().getFullYear());
  expect(app['title']()).toBe('mi_app_03');
  expect(app.subTitle()).toBe('DWECC & Frameworks');
});
```

```
stdout | src/app/app.spec.ts > App > should keep the title and subTitle unchanged
>>>Privado: mi_app_03
>>>Publico: IES Gregorio Prieto
>>>Publico: 2026
>>>Publico: mi_app_03 - [Signal: IES Gregorio Prieto] [2026]
```

```
> mi_app_03 src/app/app.spec.ts (2 tests | 1 failed) 20ms
> App (2)
  ✓ should create the app 15ms
  ✗ should keep the title and subTitle unchanged 5ms
```

Failed Tests 1

```
FAIL mi_app_03 src/app/app.spec.ts > App > should keep the title and subTitle unchanged
AssertionError: expected 'IES Gregorio Prieto' to be 'DWECC & Frameworks' // Object.is equality
```

```
Expected: "DWECC & Frameworks"
Received: "IES Gregorio Prieto"
```

Durante la ejecución de los tests de componentes, Angular no utiliza un navegador real (Chrome, Firefox, etc.), sino un entorno simulado basado en JSDOM.

JSDOM es una implementación en JavaScript del DOM que permite crear y manipular HTML, atributos, estilos y eventos como si estuviéramos en un navegador, pero sin interfaz gráfica.

Gracias a JSDOM, en los tests podemos acceder a cualquier elemento, usar `querySelector`, leer clases CSS, atributos o texto renderizado, y comprobar el comportamiento de la vista, todo ello de forma rápida, aislada y sin depender del navegador real del usuario.

Arreglar el test anterior y ahora añade otro test nuevo:

```
...
it('should return formatted title, subtitle and year', () => {
  const fixture = TestBed.createComponent(App);
  const app = fixture.componentInstance;

  const result = app.showTitleAndsubTitle();

  expect(result).toContain('DWEC & Frameworks');
});
```

Ahora prueba este test:

```
...
it('should render a router-outlet inside section.content', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const app = fixture.componentInstance;
  const compiled = fixture.nativeElement as HTMLElement;

  console.log('>>>Render:', compiled.innerHTML);

  // router-outlet existe
  const routerOutlet = compiled.querySelector('router-outlet');
  expect(routerOutlet).toBeTruthy();

  // router-outlet está dentro de section.content
  const contentSection = compiled.querySelector('section.content');
  console.log('>>>Render section.content:', contentSection?.innerHTML);
  console.log('>>>Render class section.content:',
    contentSection?.classList.value);

  expect(contentSection).toBeTruthy();
  expect(contentSection?.querySelector('router-outlet')).toBeTruthy();
});
```

- **fixture.detectChanges()** se usa siempre que
 - accedas al HTML
 - busques elementos con `querySelector`
 - esperes que se ejecute `ngOnInit`
 - dependas de bindings (`{{ }}`, `@if`, `@for`, `signals` en `template`)

Si pruebas la clase, no hace falta `detectChanges()`.
Si pruebas la vista, sí hace falta.

```
fixture.detectChanges():
- se llama JUSTO ANTES de comprobar el HTML.
- después de un cambio.
```

Y otros dos más:

```
...
it('should render header with class brand-name', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const compiled = fixture.nativeElement as HTMLElement;

  const header = compiled.querySelector('header');
  expect(header?.classList).toContain('brand-name');
});

it('should render brand logo and link to "/"', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const app = fixture.componentInstance;
  const compiled = fixture.nativeElement as HTMLElement;

  // existe el logo dentro del header y tiene la imagen correcta
  const logo = compiled.querySelector(
    'header.brand-name>img.brand-logo',
  ) as HTMLImageElement | null;
  expect(logo).toBeTruthy();
  expect(logo?.src).toContain('/images/logo.svg');

  // existe el enlace que envuelve header/logo
  const header = compiled.querySelector('header.brand-name');
  const link = header?.parentElement as HTMLAnchorElement | null;

  expect(link).toBeTruthy();
  expect(link?.getAttribute('href')).toBe('/');
});

it('should apply auxStyle on aux section', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();

  const compiled = fixture.nativeElement as HTMLElement;
  const section = compiled.querySelector('section.aux') as HTMLElement;

  expect(section.style.textAlign).toBe('center');
  expect(section.style.marginTop).toBe('100px');
});
```

Finalmente probamos un test negativo

```
...
it('should not render footer element', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const compiled = fixture.nativeElement as HTMLElement;

  const footer = compiled.querySelector('footer');
  expect(footer).toBeNull();
});
```

Vamos a probar ahora un evento:

```
...
it('should increase clicks when buttons are clicked', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const app = fixture.componentInstance;
  const compiled = fixture.nativeElement as HTMLElement;
  const buttons = compiled.querySelectorAll('button.btn-test');
  expect(buttons).toBeTruthy();
  expect(buttons.length).toBe(2);

  const button1=buttons[0] as HTMLButtonElement;
  const button2=buttons[1] as HTMLButtonElement;

  button1.click(); //+1
  button1.click(); //+1
  button2.click(); //+2
  button1.click(); //1
  fixture.detectChanges();

  const counter = compiled.querySelector('.click-counter') as HTMLElement;
  expect(counter.textContent).toContain('5');
  expect(app.clicks).toBe(5);
});
```

Recuerda en Angular, `@Input()` y `@Output()` se utilizan para comunicar componentes padre e hijo.

En testing, los `@Input()` se pueden establecer con `fixture.componentInstance.setInput(...)` y los `@Output()` se verifican suscribiéndose al `EventEmitter` y simulando la interacción (por ejemplo, un click).

@input

```
it('should render the input label', () => {
  const fixture = TestBed.createComponent(CompInput);

  fixture.componentInstance.setInput('label', 'Nuevo');
  fixture.detectChanges();

  const el: HTMLElement = fixture.nativeElement;
  expect(el.textContent).toContain('Nuevo');
});
```

@output

```
it('should emit increment value on click', () => {
  const fixture = TestBed.createComponent(CompOutput);
```

```
fixture.detectChanges();

const comp = fixture.componentInstance;

const received: number[] = [];
comp.increment.subscribe((v) => received.push(v));

const button: HTMLButtonElement =
  fixture.nativeElement.querySelector('button');
button.click();
button.click();

expect(received).toEqual([1, 2]);
});
```

Puedes suscribirte al EventEmitter directamente. No necesitas crear un componente padre para comprobar el @Output().

COBERTURA DE PRUEBAS

Prueba en la terminal: `ng test --coverage`

```
~/.work/fw_ejercicios_m/mi_app_03 > # P main !6 ?11 ng test --coverage
The following packages are required but were not found:
- @vitest/coverage-v8
Please install the missing packages and rerun the test command.
~/.work/fw_ejercicios_m/mi_app_03 > # P main !6 ?11
```

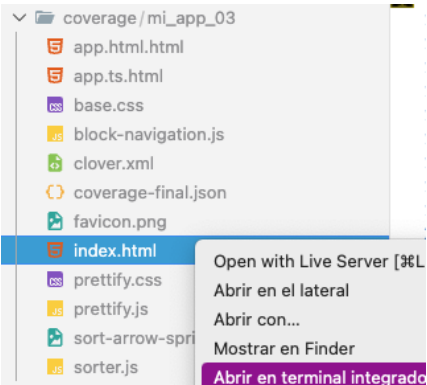
Instala los paquetes necesarios: `npm install @vitest/coverage-v8 --save-dev`

```
~/.work/fw_ejercicios_m/mi_app_03 > # P main !6 ?11 npm install @vitest/coverage-v8 --save-dev
added 11 packages, changed 10 packages, and audited 510 packages in 8s
108 packages are looking for funding
run `npm fund` for details
```

Vuelve a ejecutar la cobertura: `ng test --coverage`

No se queda esperando: `ng test --coverage --no-watch`

Al terminar, se crea una carpeta `coverage/` con un informe HTML. Abre el fichero `coverage/index.html` en el navegador para verlo.



All files

100% Statements 38/3885.71% Branches 12/14100% Functions 5/5100% Lines 22/22

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines				
app.html	100%	16/16	100%	0/0	100%	1/1	100%	12/12
app.ts	100%	14/14	85.71%	12/14	100%	4/4	100%	10/10

All files app.ts

100% Statements 14/14 85.71% Branches 12/14 100% Functions 4/4 100% Lines 18/18

Press n or j to go to the next uncovered block, b, p or k for the previous block.

```
1 import { Component, signal } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { RouterOutlet, RouterLink } from '@angular/router';
4 import { Home } from './home/home';
5
6 @Component({
7   selector: 'app-root',
8   imports: [CommonModule, RouterOutlet, RouterLink],
9   templateUrl: './app.html',
10  styleUrls: ['./app.css'],
11 })
12 export class App {
13   protected readonly title = signal('mi_app_03');
14   public subTitle = signal('');
15   public year = 2025;
16   public clicks = 0;
17
18   public auxStyle: Record<string, string> = {
19     textAlign: 'center',
20     marginTop: '100px',
21   };
22
23   constructor() {
24     this.subTitle.set('DWECC & Frameworks');
25     this.year = new Date().getFullYear();
26   }
27
28   public showTitleAndsubTitle() {
29     return `${this.title()} - ${this.subTitle()} ${this.year}`;
30   }
31
32   public incrementClicks() {
33     this.clicks++;
34   }
35 }
36
```

TEST BÁSICOS EN SERVICIOS

Vamos a crear un servicio y vamos a hacer distintos tipos de pruebas para ir entendiendo lo más importante. Para que algunas pruebas sean más interesantes, también vamos a añadir algunas modificaciones.

Construye el proyecto con `ng serve` y usando el navegador elimina las key de localStorage asociadas a las explicaciones previas.

Crea los siguientes elementos en la terminal:

```
ng generate component apply --skip-tests
ng generate service services/apply.service
ng generate enum enums/week-day
```

Añade las siguientes rutas:

```
{
  path: 'apply',
  component: Apply,
  title: 'Apply',
}
```

```
    canActivate: [authGuard],
  },
  ...
```

Amplía la interfaz: Amplía la interfaz InterfaceHouseForm:

interface-house-form.ts

```
import { WeekDay } from '../enums/week-day';

export interface InterfaceHouseForm {
  firstName: string;
  lastName: string;
  email: string;
  housingLocationId: number;
  consultaDate: Date;
  id: number;
  assignedDay?: WeekDay;
}
```

Amplía el servicio LocalStorageService y comenta el método saveApplication

localStorage.service

```
...

/*   saveApplication(application: InterfaceHouseForm): void {
    const applications = this.getAllApplications();
    applications.push(application);
    localStorage.setItem(this.APPLICATIONS_KEY,
JSON.stringify(applications));
  } */

  setAllApplications(applications: InterfaceHouseForm[]): void {
    localStorage.setItem(this.APPLICATIONS_KEY,
JSON.stringify(applications));
  }
  saveV2Application(application: InterfaceHouseForm): void {
    const applications = this.getAllApplications();

    //siempre insertamos al final:
    const nextId = applications.length === 0 ? 1 :
applications[applications.length - 1].id + 1;

    application.id = nextId;
    applications.push(application);
    localStorage.setItem(this.APPLICATIONS_KEY,
JSON.stringify(applications));
  }
..
```

Guarda cambio, compila y soluciona los errores para que se invoque ahora ***saveV2Application()***:

```
...
    id: 0,
  };

  // Guardar en localStorage
  this.localStorageService.saveV2Application(apply);
...

```

Creemos el enum:

week-day.ts

```
export enum WeekDay {
  Monday = 'Lunes',
  Tuesday = 'Martes',
  Wednesday = 'Miércoles',
  Thursday = 'Jueves',
  Friday = 'Viernes',
}
```

Completa el componente creado

apply.ts

```
import { Component, inject } from '@angular/core';

import { LocalStorageService } from '../services/localstorage.service';
import { ApplyService } from '../services/apply.service';
import { WeekDay } from '../enums/week-day';

@Component({
  selector: 'app-apply',
  imports: [],
  templateUrl: './apply.html',
  styleUrls: ['./apply.css'],
})
export class Apply {
  public applyService = inject(ApplyService);

  public weekDays: WeekDay[] = [
    WeekDay.Monday,
    WeekDay.Tuesday,
    WeekDay.Wednesday,
    WeekDay.Thursday,
    WeekDay.Friday,
  ];

  constructor() {

```

```
this.applyService.load();
}
}
```

apply.html

```
<section class="resultados">

  <ul>
    @for (day of weekDays; track day) {
      <li>
        Citas del {{ day }}:
        {{ applyService.getAppointmentsByDay(day).length }}
      </li>
    }
  </ul>
  <br>
  <p> <strong>Citas sin asignar:
    {{ applyService.getUnassignedAppointments().length }}
  </strong>
</p>

</section>
```

Define los métodos del servicio creado:

apply.service.ts

```
import { inject, Injectable } from '@angular/core';

import { InterfaceHouseForm } from '../interfaces/interface-house-form';
import { WeekDay } from '../enums/week-day';
import { LocalStorageService } from '../localStorage.service';

@Injectable({
  providedIn: 'root',
})
//Trabajamos con las citas en memoria hasta guardar.
export class ApplyService {
  private localStorageService = inject(LocalStorageService);

  private working: InterfaceHouseForm[] = [];

  private dirty = false; //Cambios pendientes
  constructor() {
    this.initializeTestData();
  }

  load(): void {
    this.working = this.localStorageService.getAllApplications();
    this.dirty = false;
  }
}
```

```
getAppointments(): InterfaceHouseForm[] {
    return this.working;
}

hasPendingChanges(): boolean {
    return this.dirty;
}

saveChanges(): void {
    this.localStorageService.setAllApplications(this.working);
    this.dirty = false;
}

discardChanges(): void {
    this.load();
}

addAppointment(app: InterfaceHouseForm): void {
    //crear una COPIA del objeto, no guardar la referencia
    this.working.push({ ...app });
    this.dirty = true;
}

deleteAppointmentById(id: number): boolean {
    const index = this.working.findIndex((a) => a.id === id);

    if (index === -1) {
        return false;
    }

    this.working.splice(index, 1);
    this.dirty = true;
    return true;
}

assignDay(id: number, day: WeekDay): boolean {
    const appointment = this.working.find((a) => a.id === id);
    if (!appointment) return false;

    const appointmentsThatDay = this.working.filter((a) => a.assignedDay === day
    && a.id !== id);

    if (appointmentsThatDay.length >= 2) {
        return false;
    }

    appointment.assignedDay = day;
    this.dirty = true;
    return true;
}

getAppointmentsByDay(day: WeekDay): InterfaceHouseForm[] {
    return this.working.filter((a) => a.assignedDay === day);
}

getUnassignedAppointments(): InterfaceHouseForm[] {
    return this.working.filter((a) => !a.assignedDay);
}
```

```
clear(): void {
    this.working = [];
    this.dirty = false;
}

private initializeTestData(): void {
    // Si ya hay datos, no hacemos nada
    const existing = this.localStorageService.getAllApplications();
    if (existing.length > 0) {
        return;
    }

    const baseDate = new Date(2026, 2, 6);

    const testApplications: InterfaceHouseForm[] = [
        {
            id: 1,
            firstName: 'Gon',
            lastName: 'Freeccs',
            email: 'gon@hunter.com',
            housingLocationId: 1,
            consultaDate: baseDate,
            assignedDay: WeekDay.Monday,
        },
        {
            id: 2,
            firstName: 'Killua',
            lastName: 'Zoldyck',
            email: 'killua@hunter.com',
            housingLocationId: 1,
            consultaDate: baseDate,
            assignedDay: WeekDay.Monday,
        },
        {
            id: 3,
            firstName: 'Kurapika',
            lastName: 'Kurta',
            email: 'kurapika@hunter.com',
            housingLocationId: 2,
            consultaDate: baseDate,
            assignedDay: WeekDay.Friday,
        },
        {
            id: 4,
            firstName: 'Leorio',
            lastName: 'Paradinight',
            email: 'leorio@hunter.com',
            housingLocationId: 2,
            consultaDate: baseDate,
        },
        {
            id: 5,
            firstName: 'Hisoka',
            lastName: 'Morrow',
            email: 'hisoka@hunter.com',
            housingLocationId: 3,
            consultaDate: baseDate,
        },
    ],
```

```
    },  
  ],  
  
  this.localStorageService.setAllApplications(testApplications);  
}  
}
```

Comprueba que funciona: `ng serve` y navega a: <http://localhost:4200/apply> tras autenticarte

Ahora vamos a probar paso a paso el servicio. Fichero: `apply.service.spec.ts`

ESTRATEGIA DE TESTING: Confiamos en que `LocalStorageService` ya está probado y funciona.

Esto simplifica nuestras pruebas y nos permite centrarnos en la lógica de `ApplyService`.

```
import { TestBed } from '@angular/core/testing';  
import { ApplyService } from '../apply.service';  
import { LocalStorageService } from '../localStorage.service';  
import { WeekDay } from '../enums/week-day';  
import { InterfaceHouseForm } from '../interfaces/interface-house-form';  
  
describe('ApplyService', () => {  
  //Declaramos variables que vamos a usar en cada test  
  //y que no son necesario re-crear en cada test  
  let service: ApplyService;  
  let localStorageService: LocalStorageService;  
  
  beforeEach(() => {  
    // Limpiar localStorage para evitar contaminación entre tests  
    // No confundir con localStorageService  
    localStorage.clear();  
  
    TestBed.configureTestingModule({  
      providers: [ApplyService, LocalStorageService],  
    });  
  
    localStorageService = TestBed.inject(LocalStorageService);  
    service = TestBed.inject(ApplyService);  
  
    // Recuerda: Todo ocurre sobre DOM virtual  
    localStorageService.setAllApplications([]);  
    service.clear();  
  });  
  
  it('should be created', () => {  
    console.log(service.getAppointments());  
    expect(service).toBeTruthy();  
    expect(service).toBeInstanceOf(ApplyService);  
  });  
});
```

```
it('should start with empty appointments after clear', () => {
  expect(service.getAppointments()).toHaveLength(0);
  expect(service.hasPendingChanges()).toBe(false);
});

});
```

Vamos a probar el load y que en cada prueba eliminamos los datos de ejemplo:

```
it('should load appointments from localStorage after load()', () => {
  localStorageService.setAllApplications([
    {
      id: 1,
      firstName: 'Gon',
      lastName: 'Freeccs',
      email: 'gon@hunter.com',
      housingLocationId: 1,
      consultaDate: new Date(2026, 2, 6),
      assignedDay: WeekDay.Monday,
    },
  ]);

  service.load();
  console.log(service.getAppointments());
  expect(service.getAppointments()).toHaveLength(1);
  expect(service.hasPendingChanges()).toBe(false);
});
```

Uso de FACTORY y variable fija para hacer los test más legibles:

```
describe('ApplyService', () => {
  //Declaramos variables que vamos a usar en cada test
  //y que no son necesario re-crear en cada test
  let service: ApplyService;
  let localStorageService: LocalStorageService;

  const baseDate = new Date(2026, 2, 6);

  function makeApp(id: number, assignedDay?: WeekDay): InterfaceHouseForm {
    return {
      id,
      firstName: `User${id}`,
      lastName: 'Test',
      email: `user${id}@test.com`,
      housingLocationId: 1,
      consultaDate: baseDate,
      assignedDay,
    };
  }

  beforeEach(() => {
    ...
  })
});
```


Vamos a usar esa Factory para probar una inserción en memoria:

```
it('should add appointment to working array', () => {
  const newApply = makeApp(1);
  service.addAppointment(newApply);

  const appointments = service.getAppointments();
  expect(appointments).toHaveLength(1);
  expect(appointments[0]).toEqual(newApply);
});

it('should mark dirty flag as true when adding appointment', () => {
  expect(service.hasPendingChanges()).toBe(false);
  service.addAppointment(makeApp(1));
  expect(service.hasPendingChanges()).toBe(true);
});
```

Ahora vamos a probar varias inserciones en memoria, la inserción en LocalStorage y la carga:

```
it('should allow adding multiple appointments', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));
  service.addAppointment(makeApp(3));

  expect(service.getAppointments()).toHaveLength(3);
  expect(service.getAppointments().map((a) => a.id)).toEqual([1, 2, 3]);
});

it('should load appointments from LocalStorageService', () => {
  const savedData = [makeApp(1), makeApp(2, WeekDay.Monday), makeApp(3,
    WeekDay.Friday)];
  localStorageService.setAllApplications(savedData);

  service.load();

  expect(service.getAppointments()).toHaveLength(3);
  expect(service.getAppointments()[0].id).toBe(1);
  expect(service.getAppointments()[1].assignedDay).toBe(WeekDay.Monday);
  expect(service.getAppointments()[2].assignedDay).toBe(WeekDay.Friday);
});
```

Ahora vamos a probar la asignación de días:

```
it('should return false when assigning day to non-existent appointment', () => {
  const result = service.assignDay(999, WeekDay.Monday);
  expect(result).toBe(false);
});

it('should assign day when there are fewer than 2 appointments on that day', () => {
  service.addAppointment(makeApp(1)); // Sin día asignado
  service.addAppointment(makeApp(2, WeekDay.Monday)); // 1 cita en lunes

  const result = service.assignDay(1, WeekDay.Monday);

  expect(result).toBe(true);
});
```

```
expect(service.getAppointmentsByDay(WeekDay.Monday)).toHaveLength(2);

const modified = service.getAppointments().find(a => a.id === 1);
expect(modified?.assignedDay).toBe(WeekDay.Monday);
});

it('should NOT assign day when there are already 2 appointments on that day',
() => {
  service.addAppointment(makeApp(1, WeekDay.Monday));
  service.addAppointment(makeApp(2, WeekDay.Monday));
  service.addAppointment(makeApp(3)); // Sin asignar

  const result = service.assignDay(3, WeekDay.Monday);

  expect(result).toBe(false);
  expect(service.getAppointmentsByDay(WeekDay.Monday)).toHaveLength(2);

  const unchanged = service.getAppointments().find(a => a.id === 3);
  expect(unchanged?.assignedDay).toBeUndefined();
});

it('should return only unassigned appointments', () => {
  service.addAppointment(makeApp(1, WeekDay.Friday));
  service.addAppointment(makeApp(2)); // Sin asignar
  service.addAppointment(makeApp(3)); // Sin asignar
  service.addAppointment(makeApp(4, WeekDay.Monday));

  const unassigned = service.getUnassignedAppointments();

  expect(unassigned).toHaveLength(2);
  expect(unassigned.map((a) => a.id)).toEqual([2, 3]);
  expect(unassigned.every((a) => a.assignedDay === undefined)).toBe(true);
});
```

Ahora vamos a probar el borrado:

```
it('should return false when deleting non-existent id', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));

  const result = service.deleteAppointmentById(999);
  expect(result).toBe(false);
  expect(service.getAppointments()).toHaveLength(2);
});

it('should delete appointment and return true when id exists', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));
  service.addAppointment(makeApp(3));

  const result = service.deleteAppointmentById(2);

  expect(result).toBe(true);
  expect(service.getAppointments()).toHaveLength(2);
  expect(service.getAppointments().map((a) => a.id)).toEqual([1, 3]);
});
```

```
expect(service.hasPendingChanges()).toBe(true);
});

it('should handle deleting from single-element list', () => {
  service.addAppointment(makeApp(1));
  const result = service.deleteAppointmentById(1);

  expect(result).toBe(true);
  expect(service.getAppointments()).toHaveLength(0);
});
```

Estas son una serie de pruebas esenciales, pero aún nos quedan aspectos del servicio sin probar:

- Más verificaciones de que el cambio de dry se hace correctamente: `hasPendingChanges()`
- Caso límite: eliminar el primero de varios
- Caso límite: eliminar el último de varios
- Reasignar al mismo día no debe rechazarse
- Cambiar una cita de lunes a viernes
- Distribuir citas en múltiples días
- Probar asignación a cada día de la semana
- `getAppointmentsByDay` con múltiples citas
- `getAppointmentsByDay` cuando no hay nada
- Guardar múltiples tipos de cambios
- Hacer cambios y descartarlos
- `clear()` no debe tocar el storage
- Flujo completo de usuario
- ...

Ahora vamos a probar el uso de espías que vigilan si se llamó a un método. El spy NO impide que el método real se ejecute, solo lo observa.

```
it('should call setAllApplications when saveChanges is executed', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));

  // vi.spyOn(objeto, 'nombreMétodo') crea el espía
  const spy = vi.spyOn(localStorageService, 'setAllApplications');

  service.saveChanges();

  expect(spy).toHaveBeenCalled(); // ¿Se llamó?
  expect(spy).toHaveBeenCalledTimes(1); // ¿Cuántas veces?

  // IMPORTANTE: Siempre restaurar el spy después de usarlo
  spy.mockRestore();
});

it('should call setAllApplications with correct data', () => {
  const app1 = makeApp(1);
  const app2 = makeApp(2, WeekDay.Monday);
  service.addAppointment(app1);
```

```
service.addAppointment(app2);

const spy = vi.spyOn(localStorageService, 'setAllApplications');

service.saveChanges();

// Verificar CON QUÉ argumentos se llamó
expect(spy).toHaveBeenCalledWith([app1, app2]);

// Otra forma con matchers:
expect(spy).toHaveBeenCalledWith(
  expect.arrayContaining([
    expect.objectContaining({ id: 1 }),
    expect.objectContaining({ id: 2, assignedDay: WeekDay.Monday }),
  ]),
);

spy.mockRestore();
});

it('should NOT call setAllApplications when only adding appointments', () => {
  const spy = vi.spyOn(localStorageService, 'setAllApplications');

  // Solo agregar en memoria, NO guardar
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));

  expect(spy).not.toHaveBeenCalled();

  spy.mockRestore();
});
```

Ahora vamos a probar otra utilidad de spy:

- Vamos a interceptar un método real de una dependencia (en este caso LocalStorageService) y controlar qué devuelve, sin ejecutar su implementación original ([combinando uso de spy y uso básico de mock](#)).

De esta forma, aislamos completamente ApplyService y podemos comprobar cómo reacciona ante distintos escenarios simulados, como recibir datos concretos o un array vacío, sin depender del comportamiento real del almacenamiento.

```
it('should load mocked data using spyOn (simulated return)', () => {
  const fakeData = [makeApp(100, WeekDay.Monday), makeApp(200,
    WeekDay.Friday)];

  // Espiamos el método real y forzamos su retorno
  const spy = vi.spyOn(localStorageService,
    'getAllApplications').mockReturnValue(fakeData);

  service.load();
```

```
expect(spy).toHaveBeenCalled();

expect(service.getAppointments()).toHaveLength(2);
expect(service.getAppointments().map((a) => a.id)).toEqual([100, 200]);
spy.mockRestore();
});

it('should simulate empty storage using spy', () => {
  const spy = vi.spyOn(localStorageService,
    'getAllApplications').mockReturnValue([]);

  service.load();

  expect(service.getAppointments()).toHaveLength(0);
  spy.mockRestore();
});
```

MOCKS

En este apartado vamos a probar el servicio `HousingService`. Este servicio realiza peticiones asíncronas mediante `fetch`, por lo que vamos a utilizar mocks para simular todas las dependencias externas (en este caso, la llamada HTTP).

Nota: No vamos a probar: `submitApplication()`

El objetivo es aislar completamente el servicio, evitando depender de la red o de APIs reales. Los **mocks nos permiten controlar exactamente qué devuelve fetch, simular errores y verificar cómo responde nuestro código en cada escenario**.

De esta forma, probamos únicamente la lógica interna del servicio. En pruebas anteriores podríamos haber aplicado la misma técnica para simular `LocalStorageService`, en lugar de asumir que ya funcionaba correctamente.

No queremos probar que `fetch` funciona. Queremos probar que:

- Se llama a `fetch`.
- Se procesa correctamente el `.json()`.
- Se devuelve `[]` cuando la API devuelve `null`.
- Se maneja correctamente la respuesta.

Por eso simulamos fetch. Recuerda un Mock:

- REEMPLAZA completamente el servicio/método
- **No ejecuta el código real**
- Tú controlas qué retorna

¿Cuándo se usan?

- Cuando la dependencia NO está probada
- Cuando quieres simular errores
- Cuando la dependencia es muy lenta (BD, API)
- Cuando quieres AISLAR completamente el servicio

Crea el correspondiente fichero `housing.service.spec.ts` con este contenido:

```
import { TestBed } from '@angular/core/testing';
import { HousingService } from '../housing.service';
import { HousingLocationInfo } from '../housinglocation';

describe('HousingService', () => {
  //Declaramos variables que vamos a usar en cada test
  //y que no son necesario re-crear en cada test

  let service: HousingService;

  // Helper: crea una Response "fake" mínima para fetch
  // fetch() devuelve un objeto Response que distintos campos
  // Aquí estamos construyendo una versión mínima simulada
  function mockFetchResponse<T>(opts: { ok?: boolean; status?: number; json: T })
  {
    const { ok = true, status = 200, json } = opts;

    return {
      ok,
      status,
      json: vi.fn().mockResolvedValue(json),
    } as unknown as Response;
  } //mockResolvedValue simula una promesa resuelta

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [HousingService],
    });

    service = TestBed.inject(HousingService);
  });

  afterEach(() => {
    vi.restoreAllMocks();
    //Muy importante en tests con mocks:
    //Restauramos todos los spies después de cada test
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
    expect(service).toBeInstanceOf(HousingService);
  });

  it('getAllHousingLocations() should return locations from fetch().json()',
  async () => {
    const locationsFake: HousingLocationInfo[] = [
```

```
    { id: 1, name: 'House 1' } as any,
    { id: 2, name: 'House 2' } as any,
  ];

  //estamos reemplazando un fetch REAL por uno simulado.
  //globalThis.fetch = la función fetch nativa del navegador

  const fetchSpy = vi
    .spyOn(globalThis, 'fetch')
    .mockResolvedValue(mockFetchResponse({ json: locationsFake }));

  const result = await service.getAllHousingLocations();

  expect(fetchSpy).toHaveBeenCalledTimes(1);
  expect(result).toEqual(locationsFake);
});

it('getAllHousingLocations() should return [] when json is null/undefined',
  async () => {
    vi.spyOn(globalThis, 'fetch').mockResolvedValue(
      mockFetchResponse({ whatever: string } | null>({
        json: null,
      })),
    );

    const result = await service.getAllHousingLocations();

    expect(result).toEqual([]);
  });

it('getAllHousingLocations() should throw error when fetch fails', async () => {
  // Simular error de red
  vi.spyOn(globalThis, 'fetch').mockRejectedValue(new Error('Network error'));

  await expect(service.getAllHousingLocations()).rejects.toThrow('Network
error');
});

it('getHousingLocationById() should call fetch with ?id= and return first
element', async () => {
  const id = 7;
  const apiResult: HousingLocationInfo[] = [{ id, name: 'Only one' } as any];

  const fetchSpy = vi
    .spyOn(globalThis, 'fetch')
    .mockResolvedValue(mockFetchResponse({ json: apiResult }));

  const result = await service.getHousingLocationById(id);

  expect(fetchSpy).toHaveBeenCalledTimes(1);
  expect(String(fetchSpy.mock.calls[0][0])).toContain(`?id=${id}`);
  //calls: Es un array que guarda TODAS las llamadas al spy.
  //fetchSpy.mock.calls[0][0] = primer argumento de la primera llamada

  expect(result).toEqual(apiResult[0]);
});

it('getHousingLocationById() should return {} when API returns empty array',
  async () => {
```

```
vi.spyOn(globalThis, 'fetch').mockResolvedValue(
  mockFetchResponse<HousingLocationInfo[]>({ json: [] }),
);

const result = await service.getHousingLocationById(999);

expect(result).toEqual({});
});

it('getHousingLocationById() should build URL with id parameter', async () => {
  const fetchSpy = vi
    .spyOn(globalThis, 'fetch')
    .mockResolvedValue(mockFetchResponse({ json: [] }));

  await service.getHousingLocationById(42);

  expect(fetchSpy).toHaveBeenCalledWith('https://dwec-fw-gp.vercel.app/api/houses?id=42');
});
});
```

Cuando probamos código asíncrono debemos usar `async/await` en el test. Si no usamos `await`, el test puede finalizar antes de que la promesa se resuelva y dar falsos positivos.

Finalmente si queremos probar un método que tiene `try / catch` en función del tipo de respuesta:

```
it('getAllHousingLocations2() should return [] when response is not ok', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValue(
    mockFetchResponse({
      ok: false,
      status: 500,
      json: null,
    }),
  );

  const result = await service.getAllHousingLocations2();

  expect(result).toEqual([]);
});
```

Tipos de mocks en Vitest

a. `mockReturnValue` → Funciones síncronas. Para funciones normales (no `async`):

```
vi.spyOn(service, 'metodoNormal').mockReturnValue('resultado');
```


b. `mockResolvedValue` → Funciones async que resuelven. Para funciones que devuelven `Promise.resolve(...)`:

```
vi.spyOn(service, 'metodoAsync').mockResolvedValue('resultado');
```

c. `mockRejectedValue` → Funciones async que fallan. Para simular errores en promesas:

```
vi.spyOn(service, 'metodoAsync').mockRejectedValue(new Error('Falló'));
```

Si el servicio usara Angular `HttpClient` en lugar de `fetch`, no mockearíamos `fetch`. En Angular existe una herramienta específica para esto:

```
HttpClientTestingModule  
HttpTestingController
```



4. Creación de componentes

Generar componentes (ng generate component)
Comunicación entre componentes (Input y Output)
Ciclo de vida de los componentes

5. Templates y Data Binding

Interpolación ({{ }})
Property Binding ([])
Event Binding (())
Two-way Data Binding ([({}]))

6. Directivas

Estructurales (*ngIf, *ngFor)
Atributo (ngClass, ngStyle)
Crear directivas personalizadas (opcional)

7. Servicios e Inyección de Dependencias

Crear servicios (ng generate service)
Uso de @Injectable
Inyectar servicios en componentes
Comunicación entre componentes con servicios

8. HTTP y consumo de APIs

HttpClientModule
Realizar peticiones GET y POST
Manejo de Observables
Mostrar datos en pantalla

9. Ruteo

Configuración básica de rutas
Navegación entre componentes
Rutas con parámetros
Lazy Loading (opcional)

10. Formularios

Formularios reactivos
Validaciones básicas
Envío de formularios